

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
«САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ,
МЕХАНИКИ И ОПТИКИ»**

Факультет безопасности информационных технологий

**Дисциплина:
«Информационная безопасность баз данных»**

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ №5

Шифрование в БД

Выполнил:

Студент гр. N3248

Жук Анастасия Валерьевна



Проверил:

Салихов Максим Русланович

Санкт-Петербург

2025 г.

Цель работы: освоить методы криптографической защиты данных в PostgreSQL, включая хеширование паролей и шифрование столбцов таблиц, а также изучить практическое применение расширения pgcrypto для обеспечения конфиденциальности хранимой информации.

Задачи:

1. Изучить хеширование паролей.
2. Освоить симметричное шифрование данных.
3. Сравнить подходы к защите данных.
4. Проанализировать безопасность методов.

Ход работы

```
postgres=# \dx
Active code page: 65001
      Список установленных расширений
  Имя   | Версия | Схема   | Описание
-----+-----+-----+-----
plpgsql | 1.0    | pg_catalog | PL/pgSQL procedural language
(1 строка)

postgres=# create extension pgcrypto;
CREATE EXTENSION
postgres=# \dx
Active code page: 65001
      Список установленных расширений
  Имя   | Версия | Схема   | Описание
-----+-----+-----+-----
pgcrypto | 1.3    | public  | cryptographic functions
plpgsql | 1.0    | pg_catalog | PL/pgSQL procedural language
(2 строки)
```

1. Создать таблицу, в которой два столбца содержат хешированные значения, где одно из них сгенерировано с помощью алгоритма SHA-1. Показать, как можно выполнить проверку, используя данные двух хэшей.

Создаем таблицу для хранения хешированных значений:

```
CREATE TABLE hashed_data (
  id SERIAL PRIMARY KEY,
  original_text TEXT,
  sha1_hash TEXT,
  salted_hash TEXT
);
```

Вставляем данные с хешированием

```
INSERT INTO hashed_data (original_text, sha1_hash, salted_hash)
VALUES (
  'secret password',
  encode(digest('secret password', 'sha1'), 'hex'),
  crypt('secret password', gen_salt('bf'))
);
```

SHA-1 — криптографическая хеш-функция, преобразующая данные в 160-битный (40-символьный) хеш. Использовался для:

- Проверки целостности файлов
- Хеширования паролей (устарел)
- Цифровых подписей (не рекомендуется)

Проблемы:

- Уязвимость к коллизиям (разные данные могут давать одинаковый хеш). В 2017 году Google продемонстрировал реальную коллизию.
- Уязвим к брутфорсу (можно перебирать миллиарды хешей/сек).
- Идентичные пароли имеют одинаковый хеш.

Процесс проверки состоит из 3 шагов:

1. Получение исходных данных;
2. Вычисление хэша;
3. Сравнение с хранимым хэшем.

salted_hash – соленый хэш с использованием алгоритма Blowfish.

Соль — это случайные данные, которые:

- Добавляются к паролю перед хешированием;
- Уникальны для каждого пользователя;
- Хранятся открыто вместе с хэшем.

Алгоритм **Blowfish** адаптирован для хеширования паролей как bcrypt с ключевыми особенностями:

1. Замедленное хеширование:
 - a. Искусственно медленный алгоритм (занимает ~100мс),
 - b. Защита от перебора (брутфорс-атаки),
 - c. Количество раундов можно настраивать (стоимость).
2. Автоматическое хранение соли.

Когда пользователь вводит пароль, система:

1. Достает соль из сохраненного хэша,
2. Хеширует введенный пароль с этой солью,
3. Сравнивает с хранимым хэшем.

Функция **crypt()** автоматически извлекает соль из **salted_hash**.

```

postgres=# CREATE TABLE hashed_data (
postgres(#      id SERIAL PRIMARY KEY,
postgres(#      original_text TEXT,
postgres(#      sha1_hash TEXT,
postgres(#      salted_hash TEXT
postgres(# );
CREATE TABLE
postgres=# INSERT INTO hashed_data (original_text, sha1_hash, salted_hash)
postgres-# VALUES (
postgres(#      'secret password',
postgres(#      encode(digest('secret password', 'sha1'), 'hex'),
postgres(#      crypt('secret password', gen_salt('bf')))
postgres(# );
INSERT 0 1

```

Проверяем хэш SHA-1

```

SELECT
    id,
    original_text,
    sha1_hash = encode(digest('secret password', 'sha1'), 'hex') AS sha1_match
FROM hashed_data;

```

```

postgres=# SELECT
postgres-#      id,
postgres-#      original_text,
postgres-#      sha1_hash = encode(digest('secret password', 'sha1'), 'hex') AS sha1_match
postgres-# FROM hashed_data;
Active code page: 65001
 id | original_text | sha1_match
----+-----+-----
  1 | secret password | t
(1 строка)

```

Функция **digest()**: вычисляет SHA-1 хэш от строки 'secret password' и возвращает бинарное значение хэша (нечитаемый формат).

Функция **encode()**: конвертирует бинарный хэш в шестнадцатеричную (hex) строку.

После чего происходит сравнение вычисленного hex-хэша с хранимым значением в колонке sha1_hash в таблице. В sha1_match записывается результат сравнения: true или false.

Проверяем salted hash

```

SELECT
    id,
    original_text,
    salted_hash = crypt('secret password', salted_hash) AS password_match
FROM hashed_data;

```

```

postgres=# SELECT
postgres=#      id,
postgres=#      original_text,
postgres=#      salted_hash = crypt('secret password', salted_hash) AS password_match
postgres=# FROM hashed_data;
Active code page: 65001
 id | original_text | password_match
-----+-----+-----
  1 | secret password | t
(1 строка)

```

2. Создать таблицу, в которой данные имеют байтовый тип. Зашифровать этот столбец и показать, как пользователь может расшифровать данные во время обычного select-запроса к зашифрованному столбцу.

Создаем таблицу с зашифрованными данными:

```

CREATE TABLE encrypted_data (
  id SERIAL PRIMARY KEY,
  secret_data BYTEA,
  encryption_key TEXT
);

```

Вставляем зашифрованные данные (используем ключ 'my_secret_key'):

```

INSERT INTO encrypted_data (secret_data, encryption_key)
VALUES (
  pgp_sym_encrypt('This is confidential information', 'my_secret_key'),
  'my_secret_key'
);

```

```

postgres=# CREATE TABLE encrypted_data (
postgres=#      id SERIAL PRIMARY KEY,
postgres=#      secret_data BYTEA,
postgres=#      encryption_key TEXT
postgres=# );
CREATE TABLE
postgres=# INSERT INTO encrypted_data (secret_data, encryption_key)
postgres=# VALUES (
postgres=#      pgp_sym_encrypt('This is confidential information', 'my_secret_key'),
postgres=#      'my_secret_key'
postgres=# );
INSERT 0 1

```

Выполняется **симметричное шифрование данных** помощью расширения **pgcrypto**.

Функция **pgp_sym_encrypt()**: шифрует текст 'This is confidential information' с помощью алгоритма AES (по умолчанию), использует ключ 'my_secret_key' для шифрования и возвращает бинарные данные (BYTEA), которые нельзя прочитать без ключа.

Пытаемся прочитать без расшифровки (видим только бинарные данные):

```

SELECT id, secret_data FROM encrypted_data;

```

```

postgres=# SELECT id, secret_data FROM encrypted_data;
Active code page: 65001
 id |
-----+-----
 1 | \xc30d0407030297bed02bfed2d0b272d25101f72022f7541c00bc913180ba5f8fd5811d19f8ea710b42c75f7b668b43da8dac2762d69930974531f0c8910b672042907de54cfc83e7ba5e9da54e999f6e4366fad9067075347e583993dfd9db46cec9
(1 строка)

```

Расшифровываем данные при запросе:

```

SELECT
  id,
  pgp_sym_decrypt(secret_data::bytea, encryption_key) AS decrypted_data
FROM encrypted_data;

```

```

postgres=# SELECT
postgres-#      id,
postgres-#      pgp_sym_decrypt(secret_data::bytea, encryption_key) AS decrypted_data
postgres-# FROM encrypted_data;
Active code page: 65001
 id | decrypted_data
-----+-----
 1 | This is confidential information
(1 строка)

```

Попытка расшифровки с неверным ключом (должна вызвать ошибку):

```

SELECT
  id,
  pgp_sym_decrypt(secret_data::bytea, 'wrong_key') AS decrypted_data
FROM encrypted_data;

```

```

postgres=# SELECT
postgres-#      id,
postgres-#      pgp_sym_decrypt(secret_data::bytea, 'wrong_key') AS decrypted_data
postgres-# FROM encrypted_data;
ОШИБКА: Wrong key or corrupt data

```

Вывод

В ходе лабораторной работы были изучены ключевые методы защиты данных в PostgreSQL:

1. Хеширование (SHA-1, crypt()):
 - a. Используется для паролей (необратимое преобразование).
 - b. crypt() с солью (bf – Blowfish) безопаснее SHA-1.
2. Симметричное шифрование (pgp_sym_encrypt):
 - a. Применяется для конфиденциальных данных (номера карт, персональные данные).
 - b. Требует безопасного хранения ключа (не в БД!).

Ключевые проблемы:

- SHA-1 уязвим к коллизиям и брутфорсу.
- Хранение ключей шифрования в открытом виде снижает безопасность.